AARHUS UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Term:	FALL 2021
Examination:	Declarative Programming
Date:	18 January 2022
Duration:	3 hours

REMEMBER to state name and study number on all pages of your submission.

- This is the exam for the lecture Declarative Programming. It consists of 18 questions. You have 3 hours to answer all of them. Strategically, you should first scan the exam paper quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are questions that require writing a calculation or the result of a calculation, and questions that require you to write Ocaml or Prolog code and drawings, in particular. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt what is expected, please ask!**
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.

Question 1 **[6]**

Recall the signatures of the functions (=) and (+)

```
val (=) : 'a -> 'a -> bool
val (+) : int -> int -> int
```

What are the signatures of the functions `e`, `f` and `g`?

```
let e x y = x = y
let f x y z = x + y + z
let g x y z = x + y = z
```

Answer:

`val e :` a' -> a' -> bool

`val f :` int -> int -> int -> int

`val g :` int -> int -> int -> bool

Question 2 **[4]**

(a) Define a function with signature `val add2 : int -> int` that adds `2` to its argument, that is,

```
3 = add2 1
4 = add2 2
```

Answer: `let` add2 x = x + 2

(b) Define a function with signature `val addn : int -> (int -> int)` that returns a function that adds `n` to its argument, that is,

```
3 = addn 2 1
4 = addn 2 2
4 = addn 3 1
5 = addn 3 2
```

Answer: `let` addn n x = x + n

Question 3 **[6]**

Complete the following recursive function `hasone xs` so that it returns `true` if `xs` contains the value `1` and `false` otherwise.

```
let rec hasone xs =
  match xs with
  1::_          -> true
  _::xs'        -> hasone xs'
  []            -> false
```

Question 4 **[6]**

You are given the following recursive implementation of function `countne xs` that counts the number of non-empty lists contained in the list `xs`.

```
let rec countne xs =
  match xs with
    (_::_)::xs' -> 1 + countne xs'
  | []::xs'     -> countne xs'
  | []          -> 0
```

Complete the following tail recursive function `countne_tailrec xs []` that counts the number of non-empty lists contained in the list `xs`. Hint: The second argument is an accumulator.

```
let rec countne_tailrec xs ac =
  match xs with

    (_::_)::xs' -> countne_tailrec xs' (ac + 1)

  | []::xs'     -> countnte_tailrec xs' ac

  | []          -> ac
```

Question 5 **[4]**

Function `filter` is defined as

```
let rec filter f xs =
  match xs with
    [] -> []
  | x::xs' -> if f x then x::filter f xs' else filter f xs'
```

(a) Using function `filter`, implement function `filter_0` that removes all integer values smaller than `1` from a list of integer values.
Answer:

```
let filter_0 = filter (fun x -> x >= 1)
```

(b) Using function `filter`, implement function `filter_a` that removes all integer values smaller than some value `a` from a list of integer values.
Answer:

```
let filter_a a = filter (fun x -> >= a)
```

Question 6 **[7]**

(a) Unnest the following function `foo`

```
let foo a b =
  let bar c = a * c in
  bar (a + b)
```

Answer:

```
let bar c a = a * c
let foo a b = bar (a + b) a
```

(b) Unnest the following function `tic`

```
let tic x =
  let tac y =
    let toe z = z - y in
    x + toe 1 in
  tac (x+1)
```

Answer:

```
let toe z y = z - y
let tac y x = x + toe 1 y
let tic x = tac (x+1) x
```

Hint: Function declarations can be nested, that is, “inner” functions are declared locally to “outer” functions. Parameters to outer functions can be referred to in inner functions.
Nested functions can be unnested by declaring all “inner” functions globally before the outer function making the passed parameters explicit. The behaviours of the nested and corresponding unnested functions are indistinguishable.

Question 7 **[5]**

Given the type `nat` below

```
type nat = Z | S of nat
```

and the two functions `sum` and `shr`

```
let rec sum x y =
  match x with
    Z -> y
  | S x' -> S (sum x' y)

let rec shr x =
  match x with
    S (S x') -> S (shr x')
  | S _ -> Z
  | Z -> Z
```

We can compute

```
shr (sum (S (S Z)) (S (S (S Z)))) = S (S Z)
```

Reading the function definitions as a system of equations as we have done in the course, calculate the result of the computation by inserting the missing steps:

```
  shr (sum (S (S Z)) (S (S (S Z))))
= shr (S (sum (S Z) (S (S (S Z)))))
= shr (S (S (sum Z (S (S (S Z)))))
= shr (S (S (S (S (S Z)))))
= S (shr (S (S (S Z))))
= S (S (shr (S Z)))

= S (S Z)
```

Question 8 **[5]**

(a) Recall the definition of the function `foldr` given by

```
let rec foldr f a xs =
  match xs with
    [] -> a
  | x::xs' -> f x (foldr f a xs')
```

Using this function implement a function `sum` that adds all integer values of a list, that is,

```
0 = sum []
3 = sum [1; 2]
5 = sum [2; 1; 2]
```

Answer:

```
let sum xs =
```
foldr + 0 xs

(b) Using function `map` in addition to `foldr` where

```
let rec map f xs =
  match xs with
    [] -> []
  | x::xs' -> f x::map f xs'
```

implement a function `flatsum` that adds all integer values of a list of lists, that is,

```
0 = flatsum [[]]
0 = flatsum [[]; []]
1 = flatsum [[]; [1]]
3 = flatsum [[1]; [2]]
5 = flatsum [[2]; [1; 2]]
```

Answer:

```
let flatsum xs =
```
foldr + 0 (map (foldr + 0) xs)

Question 9 **[8]**

The following functions `even` and `odd` compute whether a list has an even or odd number of elements.

```
let rec even xs =
  match xs with
    [] -> true
  | _::xs' -> odd xs'
and odd xs =
  match xs with
    [] -> false
  | _::xs' -> even xs'
```

Considering the function definitions as a system of equations, calculate:

(a) The value of `even [1; 2; 3]`

`even [1; 2; 3]`

= odd [2; 3]

= even [3]

= odd []

= false

(b) The value of `odd [3; 2; 1]`

`odd [3; 2; 1]`

= even [2; 1]

= odd [1]

= even []

= true

Question 10 **[6]**

Consider the pattern matching algorithm match from the course

$$
\begin{array}{lll}
\mathrm{match}\ S\ u\ =\ u & & \text{if } S = \emptyset \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ u & & \text{if } (x,t) \in S \cap u \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ (u \cup \{(x,t)\}) & & \text{if } (x,t) \in S \wedge (x,s) \notin u \text{ for all } s \\
\qquad\qquad \mathrm{match}\ (S \cup \{(s_1,s'_1),\ldots,(s_k,s'_k)\}\ u & & \text{if } (c(s_1,...,s_k), c'(s'_1,...,s'_k)) \in S \wedge c = c' \\
\qquad\qquad \mathbf{fail} & & \text{otherwise}
\end{array}
$$

Let x, y and z denote variables and F, G, S and Z constants. As usual, we leave away empty argument lists, e.g., instead of $Z()$ we write Z.

Calculate:

(a) The value of $\mathrm{match}\ \{(F(x,G(y,Z(y))),F(S,G(z,Z(z))))\}\ \emptyset$

$\mathrm{match}\ \{(F(x,G(y,Z(y))),F(S,G(z,Z(z))))\}\ \emptyset$

= match {(x,S), (G(y,Z(y)),G(z,Z(z)))} Ø

= match {(G(y,Z(y)),G(z,Z(z)))} {(x,S)}

= match {(y,z), (Z(y),Z(z))} {(x,S)}

= match {(Z(y),Z(z)))} {(x,S), (y,z)}

= match {y,z} {(x,S), (y,z)}

= match Ø {(x,S), (y,z)}

= {(x,S), (y,z)}

(b) The value of $\mathrm{match}\ \{(F(x,S(x)),F(G,S(Z)))\}\ \emptyset$

$\mathrm{match}\ \{(F(x,S(x)),F(G,S(Z)))\}\ \emptyset$

= match {(x,G), (S(x), S(Z))} Ø

= match {(S(x), S(Z))}, {(x,G)}

= match {(x,Z)}, {(x,G)}

= fail

Question 11 **[5]**

Using the truth table method, determine whether the formula

$$(P \Rightarrow Q) \Rightarrow (P \wedge \neg Q)$$

is satisfiable.

Fill in the truth table:

P	Q	$(P \Rightarrow Q) \Rightarrow (P \wedge \neg Q)$
F	F	F
F	T	F
T	F	T
T	T	F

Tick the correct answer:

☒ The formula is satisfiable.

☐ The formula is unsatisfiable.

Question 12 **[4]**

Determine whether the formula

$$\begin{array}{ll} & P \vee Q \vee R \vee S \\ \wedge & P \vee Q \vee \neg R \\ \wedge & Q \vee \neg S \\ \wedge & \neg Q \end{array}$$

is satisfiable applying the DPLL algorithm, using the rules

(UR) the 1-literal rule (aka unit clause rule),
(PR) the affirmative-negative rule (aka pure literal rule),
(SR) the splitting rule.

Fill into the boxes the formulas resulting from the rule applications indicated between the boxes.

1:

$$\begin{array}{ll} & P \vee Q \vee R \vee S \\ \wedge & P \vee Q \vee \neg R \\ \wedge & Q \vee \neg S \\ \wedge & \neg Q \end{array}$$

Applying rule **(PR)** to the formula in box 1 yields:

2:

$$\begin{array}{ll} & Q \vee \neg S \\ \wedge & \neg Q \end{array}$$

Applying rule **(UR)** to the formula in box 2 yields:

3:

$\neg S$

Applying rule **(PR)** to the formula in box 3 yields:

4:

T

Tick the correct answer:
☒ The formula is satisfiable.
☐ The formula is unsatisfiable.

Question 13 **[6]**

The following table specifies two lambda terms in the first row. Complete the empty cells in the remaining rows of the table according to the lambda terms for each column.
Remark: In your answers you may write "L" to represent "λ" if you prefer.

Lambda term	$s\ (\lambda\ x\ y\ \cdot\ y)\ z$	$\lambda\ x\ y\ \cdot\ t\ x\ y$
Name the **type** of lambda term	combination	abstraction
Rewrite the term in its fully curried form	s (λx · (λy · y)) z	λx · (λy · t x y)
Specify the **free** variables	s, z	t
Specify the **bound** variables	x, y	x, y

Question 14 **[6]**

Consider the following Prolog program:

```
person(sally).
person(tom).
person(harriet).
person(bart).

likes(sally, tom).
likes(harriet, bart).
likes(sally, bart).
likes(tom,sally).

friend(X,Y) :- likes(X,Y), likes(Y,X).

enemy(X,Y) :- person(X), person(Y), X \= Y, not(likes(X,Y)).
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog. For example:

```
?- person(X).
```

First query result when executed in Prolog:

```
X = sally.
```

(i) `?- likes(X,bart).`

First query result when executed in Prolog:

X = harriet.

(ii) `?- friend(X,Y).`

First query result when executed in Prolog:

X = sally, Y = Tom.

(iii) `?- enemy(X,Y).`

First query result when executed in Prolog:

X = sally, Y = harriet.

(b) Consider the saying:
"*The enemy of my enemy is my friend.*"

Write this definition of the *friend* relation as a Prolog rule:

```
friend(X,Z) :- enemy(X,Y),enemy(Y,Z).
```

(c) Two people X, Z are in the *same social network* if they are friends, or there exists n people $Y_1, Y_2, ..., Y_n$ such that $friend(X, Y_1)$, $friend(Y_1, Y_2)$, $\ldots$, $friend(Y_n, Z)$, for $1 \leq n$. Define the binary relation *same_network* with Prolog rules using *recursion*.

Remark: You can ignore the case that your rule will get stuck in an infinite loop if there is a cycle in the friend network. That is, it is acceptable if your rule can get stuck in an infinite loop due to such a cycle.

Hint: you'll need to define a base case rule and a recursive case rule.

```
same_network(X,Z) :- friend(X,Z).

same_network(X,Z) :- friend(X,Y), same_network(Y,Z).
```

(d) Change your rules that define the *same social network* relation so that the rules do not get stuck in an infinite loop.

Hint: you'll need to keep track of the friends that are visited, $Y_1, Y_2, ..., Y_n$.

```
same_network(X,Z,Friends) :- friend(X,Z).

same_network(X,Z,Friends) :-
 friend(X,Y), not(member(Y,Friends)), same_network(Y,Z,[Y|Friends]).
```

Question 15 **[5]**

(a) Write down a **variant** of the following Prolog fact and rule:

- `q(t(1),q(Q)).`

  Answer:

  q(t(1),q(Q0)).

- `a(B,C) :- d(B,E), f(E,C).`

  Answer:

  a(B0,C0) := d(B0,E0), f(E0,C0).

(b) Write down an **instance** of the following Prolog terms:

- `circle(centre(X,Y), radius(R))`

  Answer:

  circle(centre(0,0), radius(1))

- `a(B,C, b(B, a(E)), c(E,C))`

  Answer:

  a(1,2, b(1, a(3)), c(3,2))

(c) Given the following Prolog term and substitution, perform the substitution:

- `ins(Fa, Bf, bf(Fa)) {Bf=fA(fa), Fa=Q}`

  Answer:

  ins(Q, fA(fa), bf(Q))

Question 16 **[8]**

(a) Write the following as Prolog rules and facts in a Prolog program:

- all dogs are animals
- Henry, Sally and Tom are dogs
- Sally barks at Tom
- Henry barks at everything

Hint: You will need to implement predicates *dog*, *animal*, and *barks*. Consider how many arguments each of these predicates needs.

Answer:

```
animal(X) :- dog(X).
dog(henry).
dog(sally).
dog(tom).
bark(sally, tom).
bark(henry, _).
```

(b) Use your program to write the following questions as Prolog queries in the Prolog terminal, and to give Prolog's answer to each query.

(i) Is Henry an animal?

Prolog query:

```
?- animal(henry).
```

First query result when executed in Prolog:

```
true.
```

(ii) Who barks at the dog that barks at Tom?

Prolog query:

```
?- bark(Who, X), bark(X, tom).
```

First query result when executed in Prolog:

```
Who = henry, X = sally.
```

(iii) Does Henry bark at himself?

Prolog query:

```
?- bark(henry, henry)
```

First query result when executed in Prolog:

```
true.
```

Question 17 **[4]**

In the lecture we discussed the unification procedure, consisting of six rewriting "rules" that step-by-step transform a set of unification expressions until arriving at a "solved substitution" or a failure.

The six rules are: *swap*, *eliminate*, *decompose*, *conflict*, *occurs*, *delete*.

Apply the same procedure to unify the expressions in (a) and (b) below. Apply exactly one "rule" at each step, and specify the rule used by writing the rule name to the right side of the new unification expression set (as we did in lectures).
Remark: In your answers you can write "$=$" to represent "$\approx$" if your prefer.

(a) Unify the term `p(m(S), S, p(A, b, C))` with the term `p(L, S, p(q, S, L))`.

Answer:

$\{p(m(S), S, p(A, b, C)) \approx p(L, S, p(q, S, L))\}$	
→ {m(S) ≈ L, S ≈ S, p(A,B,C) ≈ p(q, S, L)}	decompose
{m(S) ≈ L, p(A,B,C) ≈ p(q,S,L)}	delete
{m(S) ≈ L, A ≈ q, B ≈ S, C ≈ L}	decompose
{m(S) ≈ L, A ≈ q, S ≈ B, C ≈ L}	swap
{L ≈ m(S), A ≈ q, S ≈ B, C ≈ L}	swap
{L ≈ m(B), A ≈ q, S ≈ B, C ≈ L}	eliminate
{L ≈ m(B), A ≈ q, S ≈ B, C ≈ m(B)}	eliminate

(b) Unify the term `g(A, C, g( g(A), B))` with the term `g(B, g(D), g( C, g(D)))`.

Answer:

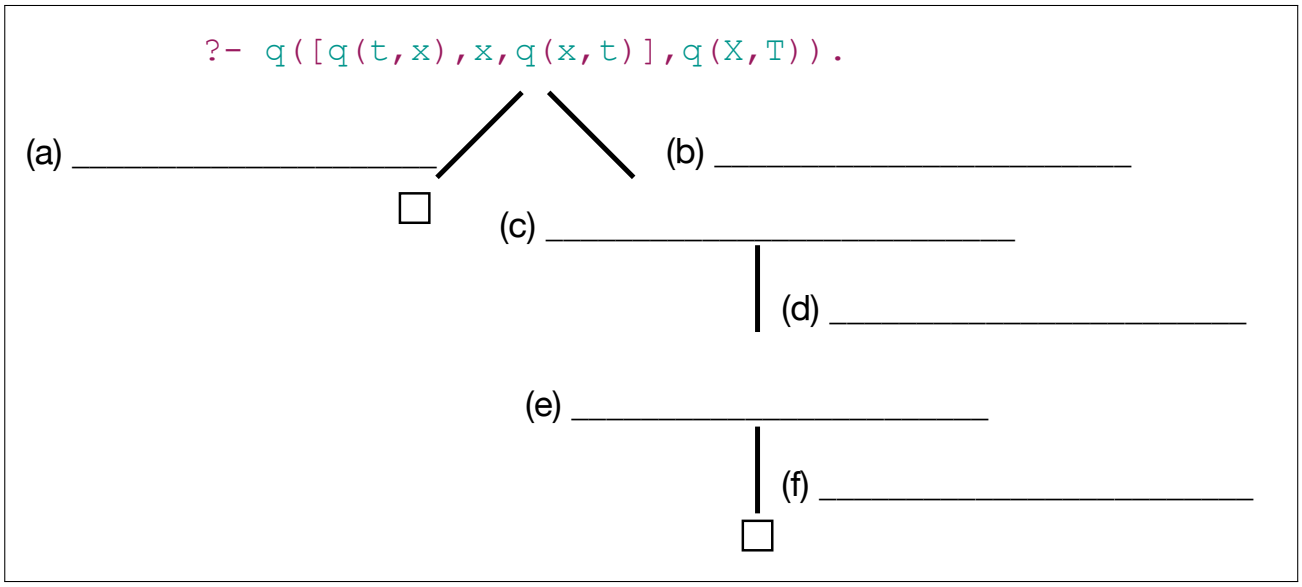
$\{g(A, C, g(g(A), B)) \approx g(B, g(D), g(C, g(D)))\}$	
→ {A ≈ B, C ≈ g(D), g(g(A),B) ≈ g(C,g(D))}	decompose
{A ≈ B, C ≈ g(D), g(A) ≈ C, B ≈ g(D)}	decompose
{A ≈ g(D), C ≈ g(D), g(A) ≈ C, B ≈ g(D)}	eliminate
{A ≈ g(D), C ≈ g(D), g(A) ≈ g(D), B ≈ g(D)}	eliminate
{A ≈ g(D), C ≈ g(D), g(g(D)) ≈ g(D), B ≈ g(D)}	eliminate
failure	occurs

Question 18 **[5]**

Consider the following Prolog rules that define the predicate `q/2`:

```
q([X|_], X).
q([_|T], X) :- q(T,X).
```

Consider the following Prolog query:

```
q([q(t,x),x,q(x,t)],q(X,T))
```

Complete the missing goals and variants at the labels (a) to (f) for this query in the following search tree (SLD-tree)